CS743 Deep Learning — Assignment 3

Complete Solutions + Full Study Guide

Convolution, Cross-Correlation, Padding, Stride, Locality, Weight Sharing, Receptive Fields, Equivariance, Pooling

This document is written in plain text maths (no LaTeX), so it renders correctly in any Markdown viewer, on PC or mobile. Matrices and calculations are in fixed-width blocks. All numerical answers were independently re-computed and checked.

How to use: read Part 1 first (it gives you the reasoning), then match your answers against Part 2 question by question. For each question the working is shown, not just the final number, because that is what earns marks.

Part 1 — The Core Theory (learn this once, use it everywhere)

1.1 Cross-correlation vs. mathematical convolution

Cross-correlation slides the filter over the input in the SAME order and takes an inner product:

```
y_i = sum over j of  k_j * x_(i+j-1)          j = 1..K
```

Mathematical convolution reverses the filter first:

```
conv(x, k) = cross_correlation(x, reverse(k))
```

So a 1-D filter `k = (k1, k2, k3)` becomes `(k3, k2, k1)` for convolution.

Important: deep-learning libraries (`Conv1d` , `Conv2d`) actually compute cross-correlation. That is why a learned layer is still called a "convolution".

Key facts:

- Correlation and convolution give the SAME result only if the filter is symmetric under

reversal, `k_j = k_(K+1-j)` (for example `(a, b, a)` , or a length-1 filter).

- For learned filters, the flip is just a relabelling of the parameters: if the optimal

correlation filter is k, the optimal convolution filter is reverse(k). The model's power is unchanged. The distinction still matters for hand computations and for interpretation.

1.2 Output size (valid / same, with stride and padding)

One dimension:

```
m = floor( (n + 2p - k) / s ) + 1
```

Two dimensions: apply the same formula separately to height and width.

```
H_out = floor( (H + 2p_h - k_h) / s_h ) + 1
W_out = floor( (W + 2p_w - k_w) / s_w ) + 1
```

- Valid: p = 0.
- Same padding: choose p so output size equals input size. With stride 1, p = (k-1)/2

(needs odd k).

- Stride s: filter jumps s pixels per step. s > 1 downsamples.
- Padding p adds 2p to the length; padded length = n + 2p.
- floor is used because an incomplete final window is discarded.

1.3 Convolution as a local inner product (pattern matching)

At output location (i, j) the layer takes the local patch P(i,j) of the image and computes

```
Y(i,j) = < P(i,j), K >  = sum over (a,b) of  K(a,b) * X(i+a-1, j+b-1)
```

- A large response means the patch "looks like" the filter.
- A constant patch of value c gives response c * (sum of filter coefficients).
- If the filter coefficients sum to zero, constant/brightness-only regions give response 0.

Such a filter responds to local CHANGE (edges, gradients), not to brightness.

1.4 Locality, weight sharing, parameters, receptive field

- Local connectivity: each output unit reads only a k x k neighbourhood.
- Weight sharing: the same filter is used at every spatial position.
- Parameters depend on k and channel counts, never on H or W.
- Receptive field (RF): the region of the original input a unit depends on.

For L stacked stride-1 k x k convolutions:

```
RF = 1 + L * (k - 1)          for k = 3:  RF = 2L + 1
```

So L = 1, 2, 3 gives RF = 3, 5, 7. Growth is LINEAR in depth.

- Stacking local operations therefore builds large-context features from small local ones

(edges -> parts -> objects), while parameter count stays independent of image size.

1.5 Master parameter formula (multi-channel)

For input H x W x C_in, filter k x k, output channels C_out:

```
params = k^2 * C_in * C_out  +  C_out
       = C_out * ( k^2 * C_in + 1 )
```

- One output channel uses one k x k x C_in filter: k^2 * C_in weights.
- C_out such filters: k^2 * C_in * C_out weights.
- One bias per output channel: C_out.
- H and W do NOT appear. That is the entire advantage of weight sharing.

1.6 Computational cost

Valid correlation of H x W with a k x k filter:

```
multiplications per output = k^2
number of outputs          = (H-k+1)(W-k+1)
total                      = k^2 (H-k+1)(W-k+1)  ~  k^2 * H * W
```

A dense (fully connected) map with N = H*W inputs and N outputs costs N^2.

```
ratio dense/conv ~ N^2 / (k^2 N) = N / k^2
```

Locality saves COMPUTE (per output is constant). Sharing saves PARAMETERS (independent of N). These are two different savings; do not mix them up.

1.7 Equivariance vs. invariance (most common trap)

Let T_d be translation by d: `(T_d x)_i = x_(i-d)` .

```
Equivariance:  F(T_d x) = T_d F(x)      output MOVES with the input
Invariance:    F(T_d x) = F(x)          output is UNCHANGED
```

A stride-1 convolution is EQUIVARIANT, not invariant.

Proof template (used in Q18, Q21):

```
(F(x))_i      = sum_j k_j * x_(i+j-1)
(F(Tx))_i     = sum_j k_j * (Tx)_(i+j-1)
              = sum_j k_j * x_(i+j-2)
              = sum_j k_j * x_((i-1)+j-1)
              = (F(x))_(i-1)
              = (T F(x))_i
```

Hence F(Tx) = T F(x). The reason it works: the SAME weights k_j are applied at every position (weight sharing), so shifting the input is absorbed by shifting the summation index.

Caveats:

- Finite image: shifting moves content out of one side and padding in on the other, so exact

equivariance fails NEAR THE BOUNDARY. It holds in the interior ("ignoring boundary effects").

- Stride s > 1: the output is sampled on a grid of spacing s. A 1-pixel shift need not become

a 1-output shift, so exact equivariance breaks (aliasing). Shifts by multiples of s are fine.

- Pointwise (elementwise) nonlinearities preserve equivariance:

`sigma(T z) = T sigma(z)` , so ReLU/sigmoid/tanh do not break it.

1.8 Pooling

- Max pooling: takes the maximum in each window -> strongest activation, local shift tolerance.
- Average pooling: takes the mean -> overall activation level, smoother.
- Both have ZERO trainable parameters.
- Output size (no padding): floor((n - k)/s) + 1. A 2x2, stride-2 pool halves each side.
- Pooling is many-to-one: it destroys exact position information. It is NOT exactly

translation equivariant to 1-pixel shifts; it gives only LOCAL, approximate insensitivity.

1.9 Ten mistakes that cost marks

1. Wrong indexing: the assignment uses x_(i+j-1), so the first window starts at x_1.
2. Forgetting to flip the filter when convolution (not correlation) is asked.
3. Using n-k instead of n-k+1 for the valid output length.
4. Wrong padding arithmetic: padded length is n + 2p, and k is subtracted once.
5. Using ceiling instead of floor for stride.
6. Confusing weights with biases: one bias per OUTPUT CHANNEL.
7. Putting H or W into a convolutional parameter count. They never appear.
8. Saying pooling has trainable parameters. It has zero.
9. Saying equivariance means "unchanged". That is invariance.
10. Claiming receptive field grows exponentially. For stride-1 stacks it grows linearly, 2L+1.

Part 2 — Solutions to All 38 Questions

Question 1 — 1-D valid cross-correlation

Given:

```
x = (2, 1, 3, 0, 4)
k = (1, -1, 2)
y_i = k1*x_i + k2*x_(i+1) + k3*x_(i+2)
```

(a) Number of outputs: n - k + 1 = 5 - 3 + 1 = 3.

(b) y1 — window (x1, x2, x3) = (2, 1, 3):

```
y1 = 1*2 + (-1)*1 + 2*3 = 2 - 1 + 6 = 7
```

(c) y2 — window (x2, x3, x4) = (1, 3, 0):

```
y2 = 1*1 + (-1)*3 + 2*0 = 1 - 3 + 0 = -2
```

(d) Complete output. y3 uses (x3, x4, x5) = (3, 0, 4):

```
y3 = 1*3 + (-1)*0 + 2*4 = 3 - 0 + 8 = 11
y  = (7, -2, 11)
```

(e) Entries used for y3: x3 = 3, x4 = 0, x5 = 4.

(f) Why "local": every output depends on only 3 adjacent inputs, not the whole vector. As i increases by 1, the window slides by 1. The computation touches a bounded neighbourhood.

Note: using x_(i+j) instead of x_(i+j-1) would start at x2 and give wrong numbers.

Question 2 — Convolution vs. cross-correlation

Given:

```
x = (1, 2, 0, -1, 3)
k = (2, 1, -1)
```

(a) Reversed filter: `reverse(k) = (-1, 1, 2)`.

(b) Valid cross-correlation of x with k:

```
y1 = 2*1 + 1*2 + (-1)*0      = 2 + 2 + 0  = 4
y2 = 2*2 + 1*0 + (-1)*(-1)   = 4 + 0 + 1  = 5
y3 = 2*0 + 1*(-1) + (-1)*3   = 0 - 1 - 3  = -4
y_corr = (4, 5, -4)
```

(c) Valid convolution = cross-correlation with reverse(k) = (-1, 1, 2):

```
c1 = (-1)*1 + 1*2 + 2*0      = -1 + 2 + 0 = 1
c2 = (-1)*2 + 1*0 + 2*(-1)   = -2 + 0 - 2 = -4
c3 = (-1)*0 + 1*(-1) + 2*3   = 0 - 1 + 6  = 5
y_conv = (1, -4, 5)
```

(d) Equal? No. (4, 5, -4) is not (1, -4, 5).

(e) When are they equal? When the filter equals its own reversal, k_j = k_(K+1-j) (a palindromic filter such as (a, b, a)), or trivially a length-1 filter.

Question 3 — What distinguishes the two operations

Given `k = (k1, k2, k3)`.

(a) Cross-correlation filter: `(k1, k2, k3)` — same order.

(b) Mathematical-convolution filter: `(k3, k2, k1)` — reversed order.

(c) Precise distinction:

```
correlation:  y_i = sum_j k_j * x_(i+j-1)     (both indices increase)
convolution:  y_i = sum_j k_j * x_(i-j+1)     (one increases, one decreases)
```

Convolution is correlation with a 180-degree-rotated kernel. They are related by a fixed linear reindexing of the kernel, not identical functions of (x, k).

(d) Why the distinction fades for learned filters: the flip is a fixed, invertible relabelling of the parameter vector. If the best correlation kernel is k*, the best convolution kernel is reverse(k*), and both represent the same function class. Gradient descent can learn either. So the model's capacity is unchanged; only interpretation and hand-calculation care about the difference.

Question 4 — Sliding a length-3 filter over x

Given `x = (3, 1, 4, 1, 5, 9)` , length-3 filter, valid cross-correlation.

(a) Local patches (6 - 3 + 1 = 4 windows):

```
(3, 1, 4), (1, 4, 1), (4, 1, 5), (1, 5, 9)
```

(b) Same filter used how many times? 4 times, once per output position.

(c) Trainable parameters: 3 coefficients + 1 bias = 4.

(d) Input length 6 -> 100: No change in filter parameters (still 4). Only the number of applications grows, from 4 to 98.

(e) Weight sharing: the same coefficient values are reused at every position. The filter is not re-parameterised per location. This makes the parameter count independent of input size and produces translation equivariance.

Question 5 — 2-D valid cross-correlation

Given:

```
      [ 1  2  0 ]
X  =  [ 3  1  2 ]
      [ 0  1  4 ]

      [  1   0 ]
K  =  [ -1   2 ]
```

(a) Shape of Y: H' = 3 - 2 + 1 = 2, W' = 3 - 2 + 1 = 2. So Y is 2 x 2.

(b) Y11 — patch rows 1-2, cols 1-2:

```
patch = [ 1  2 ]
        [ 3  1 ]
Y11 = 1*1 + 0*2 + (-1)*3 + 2*1 = 1 + 0 - 3 + 2 = 0
```

(c) Y12 — patch rows 1-2, cols 2-3:

```
patch = [ 2  0 ]
        [ 1  2 ]
Y12 = 1*2 + 0*0 + (-1)*1 + 2*2 = 2 + 0 - 1 + 4 = 5
```

(d) Y21 — patch rows 2-3, cols 1-2:

```
patch = [ 3  1 ]
        [ 0  1 ]
Y21 = 1*3 + 0*1 + (-1)*0 + 2*1 = 3 + 0 + 0 + 2 = 5
```

(e) Y22 — patch rows 2-3, cols 2-3:

```
patch = [ 1  2 ]
        [ 1  4 ]
Y22 = 1*1 + 0*2 + (-1)*1 + 2*4 = 1 + 0 - 1 + 8 = 8
```

(f) Complete output:

```
      [ 0  5 ]
Y  =  [ 5  8 ]
```

Question 6 — Edge-detecting filter on a block image

Given:

```
      [ 1  1  0  0 ]
X  =  [ 1  1  0  0 ]
      [ 0  0  2  2 ]
      [ 0  0  2  2 ]

      [  1  -1 ]
K  =  [  1  -1 ]
```

Output is 3 x 3. For each 2x2 patch, response = (sum of left column) - (sum of right column).

(a) Valid cross-correlation:

```
(1,1): [1 1; 1 1]   -> (1+1) - (1+1) = 0
(1,2): [1 0; 1 0]   -> (1+1) - (0+0) = 2
(1,3): [0 0; 0 0]   -> 0
(2,1): [1 1; 0 0]   -> (1+0) - (1+0) = 0
(2,2): [1 0; 0 2]   -> (1+0) - (0+2) = -1
(2,3): [0 0; 2 2]   -> (0+2) - (0+2) = 0
(3,1): [0 0; 0 0]   -> 0
(3,2): [0 2; 0 2]   -> (0+0) - (2+2) = -4
(3,3): [2 2; 2 2]   -> (2+2) - (2+2) = 0
```

```
        [ 0   2   0 ]
X*K  =  [ 0  -1   0 ]
        [ 0  -4   0 ]
```

(b) Large-magnitude locations: only the middle column is non-zero: (1,2) = +2, (2,2) = -1, (3,2) = -4. Largest magnitude at (3,2).

(c) What local change it detects: K is constant down columns but changes across columns, so it computes a horizontal derivative. It responds to VERTICAL edges (left-right intensity change). The sign gives the direction of the change. Uniform regions give zero.

(d) Local feature detector: the response is the inner product of a local patch with a fixed template. It is large only where the patch matches the template (here a left-right gradient). The output map tells you WHERE that feature occurred.

Question 7 — Deriving the valid output shape

Input H x W, filter Kh x Kw, stride 1, no padding.

(a) Height: the filter's top row i must satisfy i + Kh - 1 <= H, so i <= H - Kh + 1. Number of valid start rows = H - Kh + 1.

(b) Width: similarly W - Kw + 1.

(c) Output shape:

```
(H - Kh + 1) x (W - Kw + 1)
```

(d) 10 x 12 input, 3 x 5 filter:

```
(10 - 3 + 1) x (12 - 5 + 1) = 8 x 8
```

(e) 28 x 28 input, 5 x 5 filter:

```
(28 - 5 + 1) x (28 - 5 + 1) = 24 x 24
```

Intuition: the filter "eats" K-1 pixels in each direction because it needs full overlap.

Question 8 — Padding with a 7 x 7 input, 3 x 3 filter, stride 1

(a) No padding: (7 - 3 + 1) x (7 - 3 + 1) = 5 x 5.

(b) One row/column of zeros on every side: padded size = (7 + 2) x (7 + 2) = 9 x 9.

(c) Resulting output: (9 - 3 + 1) x (9 - 3 + 1) = 7 x 7.

(d) Why "same" padding: p = (k-1)/2 = 1 and stride 1 give H_out = H + 2p - k + 1 = 7 + 2 - 3 + 1 = 7 = H. Output size equals input size.

(e) Why padding helps at boundaries: without padding, border pixels appear in fewer filter windows (corners in only one), so border information is under-used, the output shrinks, and behaviour changes abruptly at the edge. Padding centres the filter on border pixels, keeps the spatial size, and treats edges more uniformly.

Question 9 — 1-D output lengths

Formula: m = floor((n + 2p - k)/s) + 1.

(a) n=10, k=3, p=0, s=1: (10-3)/1 + 1 = 7 + 1 = 8. **(b)** n=10, k=3, p=1, s=1: (10+2-3)/1 + 1 = 9 + 1 = 10. **(c)** n=10, k=3, p=0, s=2: floor(7/2) + 1 = 3 + 1 = 4. **(d)** n=15, k=5, p=2, s=1: (15+4-5)/1 + 1 = 14 + 1 = 15. **(e)** n=15, k=5, p=2, s=3: floor(14/3) + 1 = 4 + 1 = 5.

Answers: (a) 8, (b) 10, (c) 4, (d) 15, (e) 5.

Notes: (b) and (d) are "same" padding (output = input). In (c) the incomplete tail is dropped, which is why floor is used.

Question 10 — 32 x 32 input, 5 x 5 filter, padding 2

Core quantity: n + 2p - k = 32 + 4 - 5 = 31.

(a) s = 1: floor(31/1) + 1 = 32 -> 32 x 32. **(b) s = 2:** floor(31/2) + 1 = 15 + 1 = 16 -> 16 x 16. **(c) s = 4:** floor(31/4) + 1 = 7 + 1 = 8 -> 8 x 8.

(d) Effect of larger stride: the filter jumps further between outputs, so the output grid is coarser and spatial resolution decreases (downsampling). Each output summarises a larger region and adjacent outputs are further apart.

(e) Does stride change filter coefficients? No. Stride changes WHERE the filter is applied, not WHAT it contains. Parameter count is independent of stride.

Question 11 — One 3 x 3 output unit

(a) It depends on 3 x 3 = 9 input locations.

(b) A dense neuron on a 100 x 100 image connects to all 10,000 pixels, so it has 10,000 weights (plus bias). The 3x3 unit has 9 (plus bias). About 1111 times fewer connections.

(c) Local connectivity: each output unit connects only to a small spatial neighbourhood of the input, not the whole input.

(d) Why natural for images: nearby pixels are strongly correlated; edges, corners, textures and blobs are local patterns. A pixel's useful information is mostly in its immediate surroundings, so global connections are wasteful.

(e) Example: a vertical or horizontal edge, a corner, a small spot/blob, or a texture patch.

Question 12 — Receptive field of stacked 3 x 3 convolutions

(a) A unit in the first output depends on a 3 x 3 region of the original input.

(b) A unit in the second output reads a 3 x 3 region of the first output. Each of those units reads a 3 x 3 region of the input. The union per side is 3 + (3 - 1) = 5:

```
first layer : each unit covers 3 input pixels
second layer: covers 3 first-layer units per side
union       : 3 + 3 - 1 = 5  ->  5 x 5 region of the original input
```

So the answer is 5 x 5.

(c) After three convolutions: 1 + 3*(3-1) = 7 -> 7 x 7.

(d) Formula after L successive 3 x 3 convolutions: RF = 2L + 1. General k x k, stride 1: RF = 1 + L*(k-1).

(e) How local operations combine: each layer takes a local neighbourhood of the previous layer's outputs; composing these nested neighbourhoods enlarges the footprint by k-1 per layer. Depth therefore builds a hierarchy from tiny patterns (edges) to larger structures (parts, objects) while every operation remains local and parameter counts stay independent of image size.

Common mistake: saying the RF doubles or grows exponentially. It grows linearly.

Question 13 — Dense vs. convolutional map on 32 x 32

N = 32*32 = 1024 pixels, one scalar output per pixel.

(a) Weights for one dense output unit: all 1024 input pixels -> 1024.

(b) Total dense weights for 32 x 32 outputs: 1024 * 1024 = 1,048,576.

(c) Shared 3 x 3 filter weights: 9.

(d) Including one bias each:

```
dense: (1024 weights + 1 bias) * 1024 units = 1,049,600
conv : 9 weights + 1 bias = 10
```

(e) Factor fewer (with biases): 1,049,600 / 10 = 104,960 (about 1.05 x 10^5). Without biases: 1,048,576 / 9 = 116,508.

(f) The two assumptions:

1. Local connectivity: each output uses only a 3x3 neighbourhood, not all 1024 pixels.
2. Weight sharing: the same 9 weights are reused at all 1024 positions.

Question 14 — Dense map from dimension d to m

(a) Weights: every output connects to every input -> m * d.

(b) Biases: one per output -> m.

(c) Total parameters: m*d + m = m(d + 1).

(d) d = 784, m = 784: 784 * 784 = 614,656 weights (615,440 with biases).

(e) Single 3 x 3 filter with bias: 9 + 1 = 10 parameters. Weight ratio: 614,656 / 9 = 68,295. With biases: 615,440 / 10 = 61,544.

(f) Why conv count does not grow with spatial positions: a dense map has a separate weight for every (input pixel, output pixel) pair, so its count is O(N^2). A convolution reuses one small filter at every position, so its count is O(k^2) regardless of N. Applications grow with N; parameters do not.

Question 15 — Multi-channel filter parameter count

Input has C_in channels; one output channel uses a separate k x k kernel for each input channel.

(a) Weights for one output channel: k^2 * C_in.

(b) Including bias: k^2 * C_in + 1.

(c) For C_out filters: C_out * (k^2 * C_in + 1) = k^2 * C_in * C_out + C_out.

(d) k = 3, C_in = 3, C_out = 16:

```
9 * 3 * 16 + 16 = 432 + 16 = 448
```

(e) Depends on image height/width? No. Only k, C_in, C_out appear.

(f) Why important: model size is decoupled from image resolution. The same layer processes 32 x 32 or 4K images with identical parameters, reducing memory, overfitting and training data needs, and it gives translation equivariance.

Question 16 — Multiplication count for valid correlation

Input H x W, filter k x k.

(a) Multiplications per output entry: k^2.

(b) Number of output entries: (H - k + 1)(W - k + 1).

(c) Total multiplications: k^2 * (H - k + 1)(W - k + 1).

(d) H = W = 32, k = 3:

```
9 * (30) * (30) = 8,100
```

(e) Compare with dense: a dense map of 30x30 outputs each connected to all 1024 inputs costs 900 * 1024 = 921,600 multiplications; if the output is also 32x32 it is 1024^2 = 1,048,576. So the convolution is roughly 100 times cheaper (8,100 vs about 10^6). Dense cost is O(N^2); conv cost is O(k^2 N).

Question 17 — Asymptotic cost, N pixels and N outputs

(a) Dense cost: each of N outputs uses all N inputs -> about N^2 multiplications.

(b) Conv cost: each of N outputs uses k^2 inputs -> about N * k^2 multiplications.

(c) Ratio: N^2 / (N k^2) = N / k^2.

(d) N = 10^6, k = 3: 10^6 / 9 = 111,111 (about 1.11 x 10^5).

(e) Why the saving is large: cost per output is constant (k^2), so total cost is LINEAR in the number of pixels, while the dense map is QUADRATIC. The gap N/k^2 grows without bound as images get larger, making dense maps infeasible for megapixel images.

Question 18 — Proof that correlation commutes with translation

Given (T x)_i = x_(i-1) and (F(x))_i = sum_j k_j * x_(i+j-1).

(a) (F(x))_i = sum_j k_j * x_(i+j-1).

(b) (F(Tx))_i = sum_j k_j * (Tx)_(i+j-1) = sum_j k_j * x_(i+j-2).

(c) Rearrange: let r = i - 1. Then i + j - 2 = r + j - 1, so

```
(F(Tx))_i = sum_j k_j * x_(r+j-1) = (F(x))_r = (F(x))_(i-1) = (T F(x))_i
```

Therefore, for all i:

```
F(T x) = T F(x)
```

(d) In words: translating the input by one position produces the same output, merely translated by one position. The filter fires the same way wherever the pattern appears, and the response moves with it.

(e) Property that makes it possible: weight sharing plus position-independent local connectivity. The same k_j are applied at every position, so the index shift in the input is absorbed by a shift of the summation index, which is exactly a shift of the output index.

Boundary caveat: the derivation assumes the shifted signal is defined everywhere. On a finite image the equality holds only away from the edges.

Question 19 — Numerical illustration of equivariance

Given x = (1, 2, 0, 1, 3, 0), k = (1, -1).

(a) Original valid correlation (y_i = x_i - x_(i+1)):

```
y1 = 1 - 2 = -1
y2 = 2 - 0 =  2
y3 = 0 - 1 = -1
y4 = 1 - 3 = -2
y5 = 3 - 0 =  3
y = (-1, 2, -1, -2, 3)
```

(b) Shift input right by one (zero fill at front, boundary ignored):

```
T x = (0, 1, 2, 0, 1, 3)
```

(c) Correlation of shifted input:

```
0-1 = -1,  1-2 = -1,  2-0 = 2,  0-1 = -1,  1-3 = -2
new output = (-1, -1, 2, -1, -2)
```

(d) Compare: the original output shifted right by one is

```
T y = ( ?, -1, 2, -1, -2 )
```

Ignoring the boundary entry, the new output (-1, 2, -1, -2) matches the shifted original (-1, 2, -1, -2) exactly.

(e) What it shows: F(Tx) = T(F(x)) away from the boundary. The response follows the pattern instead of staying fixed — the numerical signature of translation equivariance.

Question 20 — Equivariance vs. invariance

(a) "Unchanged" means invariance: F(Tx) = F(x), output values identical after the input moves. "Translated" means equivariance: F(Tx) = T F(x), the same pattern shifted by the same amount.

(b) No, they are different. Equivariance preserves the transformation in the output; invariance removes it. Convolution is equivariant, not invariant.

(c) If the object moves 5 pixels right, the feature moves 5 positions right in the output (stride 1).

(d) Equivariance is useful because a detector learned at one location automatically works at every location; no separate weights per position are needed. The output still encodes WHERE the feature is, which localisation tasks need.

Question 21 — Pointwise ReLU preserves equivariance

G(x) = ReLU(F(x)), with F equivariant.

(a) "Pointwise": the nonlinearity is applied independently to each scalar element/location; it mixes no information across spatial positions.

(b) Show ReLU(T_d z) = T_d ReLU(z):

```
[ReLU(T_d z)]_i = max(0, (T_d z)_i)
                = max(0, z_(i-d))
                = [ReLU(z)]_(i-d)
                = [T_d ReLU(z)]_i
```

(c) G remains equivariant:

```
G(T_d x) = ReLU(F(T_d x))
         = ReLU(T_d F(x))
         = T_d ReLU(F(x))
         = T_d G(x)
```

(d) Other activations? Yes. Any pointwise activation sigma satisfies sigma(T_d z) = T_d sigma(z) by the same index argument: sigmoid, tanh, leaky ReLU, GELU, absolute value, etc.

(e) Why same operation everywhere matters: position dependence would break the symmetry, because a shift would no longer commute with the map. Uniform, position-independent operations preserve spatial structure and equivariance, and allow weight sharing.

Question 22 — Finite images and boundary effects

(a) A finite image occupies a bounded grid. Shifting it moves pixels off one edge (lost) and leaves a gap at the other (must be filled), so the shifted image is not a pure shifted copy near the border.

(b) Zero padding fills the gap with artificial zeros. Near the border, filter windows include these zeros, so responses there use a different (partly fabricated) signal. Shifting changes border responses in a way that is not a pure translation.

(c) Far from the border every filter window lies fully inside the image, no padding is involved, and the shift merely reindexes the same values. The equivariance proof applies exactly.

(d) Exact equivariance is a property of the operation on an infinite or periodic domain. On finite images it is true only in the interior, so "ignoring boundary effects" flags that border/padding exceptions are being set aside.

Question 23 — Stride 2

(a) Stride 2 means the filter moves 2 pixels between consecutive output positions, so outputs are computed on every second location. This downsamples the output grid by 2 per direction.

(b) Input shifted by exactly 1 pixel — must the output shift by 1 output position? No. With stride 2 the output exists only on a grid of spacing 2. A 1-pixel shift changes which input positions are sampled; there may be no corresponding output position. The output is generally not a one-position translation.

(c) Why subsampling destroys exact equivariance to 1-pixel shifts: strided sampling discards half the positions (aliasing). A 1-pixel shift can move a feature from an even to an odd index, off the sampling lattice, so the response changes qualitatively instead of shifting. F is not a bijection on positions, so F(Tx) != T F(x) in general.

(d) Input shifted by 2 pixels (ignoring boundaries): since the sampling grid has spacing 2, a 2-pixel shift is exactly one output position, so the output translates by one position. Equivariance is restored.

(e) Contrast:

```
Stride 1: equivariant to any 1-pixel shift (interior), full resolution, exact.
Stride 2: NOT equivariant to 1-pixel shifts (aliasing); equivariant to shifts that are
          multiples of 2; halved resolution; approximate/shift-selective.
```

Question 24 — 2 x 2 max pooling, stride 2

Given:

```
      [ 1  3  2  0 ]
X  =  [ 4  6  1  2 ]
      [ 0  2  5  3 ]
      [ 1  1  2  4 ]
```

(a) Four windows (non-overlapping):

```
W11 = [ 1  3 ]    W12 = [ 2  0 ]
      [ 4  6 ]          [ 1  2 ]

W21 = [ 0  2 ]    W22 = [ 5  3 ]
      [ 1  1 ]          [ 2  4 ]
```

(b) Maximum of each: 6, 2, 2, 5.

(c) Pooled output:

```
      [ 6  2 ]
Y  =  [ 2  5 ]
```

(d) Shape: 2 x 2 (4 / 2 = 2 per side).

(e) Trainable parameters: 0. Max pooling has no learnable weights.

Question 25 — 2 x 2 average pooling, stride 2 (same X)

(a) Averages:

```
W11: (1+3+4+6)/4 = 14/4 = 3.5
W12: (2+0+1+2)/4 =  5/4 = 1.25
W21: (0+2+1+1)/4 =  4/4 = 1.0
W22: (5+3+2+4)/4 = 14/4 = 3.5
```

(b) Complete pooled output:

```
      [ 3.5  1.25 ]
Y  =  [ 1.0  3.5  ]
```

(c) Comparison: max pooling keeps only the strongest activation per region and discards the rest; average pooling keeps the mean activation per region. Max output is peakier and more robust to a strong feature shifting within a window; average output is smoother and reflects the overall level.

(d) Preserves strongest activation: max pooling.

(e) Summarises overall activation level: average pooling.

Question 26 — Pooling a 28 x 28 map twice

(a) After first 2x2 stride-2 pool: 28/2 = 14 -> 14 x 14. **(b)** After second: 14/2 = 7 -> 7 x 7. **(c)** Initial spatial values: 28 * 28 = 784. **(d)** After first: 14 * 14 = 196. **(e)** After second: 7 * 7 = 49. **(f)** Decrease factor: 784 / 49 = 16 (each side shrinks by 4, area by 16).

Question 27 — 1-D max pooling, window 2, stride 2

Given x = (1, 5, 2, 4, 3, 6).

(a) Pooled output:

```
max(1,5)=5, max(2,4)=4, max(3,6)=6  ->  (5, 4, 6)
```

(b) Translate x one position right (zero fill at front):

```
T x = (0, 1, 5, 2, 4, 3)
max(0,1)=1, max(5,2)=5, max(4,3)=4  ->  (1, 5, 4)
```

(c) Is the second result a simple translation of the first? No. (1, 5, 4) is not a shifted copy of (5, 4, 6). The pooling windows pair different inputs after the shift.

(d) Conclusion: strided max pooling is not exactly translation equivariant to 1-pixel shifts. It is equivariant only to shifts that are multiples of the stride (here, even shifts), and even then only away from boundaries.

(e) Why pooling still helps: if a feature shifts a little WITHIN a window, the maximum (or average) of that window changes little or not at all. Many-to-one summarisation gives local insensitivity / approximate invariance to small shifts.

Question 28 — "Max pooling makes a CNN perfectly translation invariant"

(a) Translation invariance would mean: F(T_d x) = F(x) for all translations. The output is IDENTICAL no matter where the content is, not merely shifted.

(b) Why the statement is too strong:

- Pooling is local: it pools within fixed windows, so a shift that moves a feature across a

window boundary or into another window changes the output.

- It is strided: outputs live on a coarse grid, so arbitrary shifts do not map cleanly.
- Boundary effects break symmetry at the edges.
- Only partial, local position information is removed. The network is not perfectly invariant.

(c) The correct weaker idea (local insensitivity): within a pooling window, small translations produce the same or nearly the same pooled value, because max/mean are translation-tolerant summaries. So pooling gives local, approximate robustness, not exact global invariance.

(d) Contrast with stride-1 convolution: a stride-1 convolution is equivariant — the output moves with the input and still encodes position. Pooling is many-to-one and discards position, giving partial invariance. Equivariance preserves the transformation; invariance removes it.

Question 29 — Inner-product view Y(i,j) = <P(i,j), K>

(a) P(i,j)(X): the local k x k patch of the image at location (i,j) — the k^2 pixel values the filter covers there, flattened into a vector for the inner product.

(b) What the inner product measures: the sum of elementwise products sum_(a,b) K(a,b) * X(i+a-1, j+b-1). It is an unnormalised similarity/projection: large and positive when the patch aligns with positive coefficients and negative when it aligns with negative ones. By Cauchy-Schwarz it is maximised when the patch is proportional to the filter.

(c) Why similar patches give a large response: if the patch matches the template, positive coefficients multiply positive pixels and negative coefficients multiply low pixels, so all terms add constructively. Mismatched patches cancel.

(d) Why the same K everywhere: weight sharing. One template is reused at every location. This reduces parameters and, because the operation is position-independent, a feature learned at one location is detected identically everywhere -> translation equivariance.

(e) Connection between local detection and sharing: the filter is a local template; the inner product at each position measures how strongly that template occurs, producing a feature/ detection map. Sharing the template across positions turns detection into a convolution: one learned local detector scanned over the whole image.

Question 30 — Interpreting two filters

Given:

$$K1 = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \qquad K2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

(a) K1: coefficients vary across columns (left +1, right -1, middle 0), constant down rows. It computes a horizontal derivative and responds strongly to VERTICAL edges (a left-to-right intensity change). Bright-left/dark-right gives positive; dark-left/bright-right gives negative.

(b) K2: coefficients vary across rows (top +1, bottom -1, middle 0), constant across columns. It computes a vertical derivative and responds to HORIZONTAL edges (a top-to-bottom intensity change). Bright-above/dark-below gives positive; reversed gives negative.

(c) Constant patch: for a patch of constant value c, the response is c * (sum of coefficients) = c * 0 = 0 for both filters. Constant regions give zero response, so uniform brightness is ignored.

(d) Why signed coefficients detect changes, not brightness: positive and negative weights cancel the DC (mean brightness) component. The response depends only on differences between neighbouring pixels — local contrast, edges, gradients — not absolute intensity.

(e) Why this is a property of THESE filters, not convolution: convolution is a generic operation; what it detects is determined entirely by the coefficients. An all-positive filter responds to brightness; another filter may detect corners or textures. "Edge detector" is a property of K1 and K2, not of convolution itself.

Question 31 — RGB patch 3 x 3 x 3

(a) Scalar entries in the patch: 3 * 3 * 3 = 27.

(b) Weights in the filter: 3 * 3 * 3 = 27.

(c) How one scalar output is obtained: multiply the 27 patch values by the 27 corresponding filter weights channel-wise and spatially, sum all 27 products, add a bias:

```
y = sum over c=1..3, a=1..3, b=1..3 of K(a,b,c) * X(a,b,c) + bias
```

(d) With bias: 27 + 1 = 28 trainable parameters.

(e) Applied at 1000 locations — 1000 different weight sets? No. The same 27 weights are reused at all 1000 locations. There are 1000 outputs but one parameter set: weight sharing.

Question 32 — General multi-channel map parameter count

Input X in R^(H x W x C_in), output Y in R^(H' x W' x C_out). Each output channel uses one k x k x C_in filter.

(a) Number of filters: C_out.

(b) Total filter weights: k^2 * C_in * C_out.

(c) Total biases: C_out (one per output channel).

(d) Total parameters: k^2 * C_in * C_out + C_out = C_out * (k^2 * C_in + 1).

(e) Why H and W do not appear: parameters describe the filter, not the number of times it is applied. H and W determine the output grid size and computational cost, but the same shared weights are used at every position. Parameter count is independent of spatial resolution.

(f) k = 3, C_in = 8, C_out = 16:

```
9 * 8 * 16 + 16 = 1152 + 16 = 1168
```

Question 33 — Dense (A) vs. shared 3x3 same-padding (B), 28 x 28

(a) Flattened input dimension: 28 * 28 = 784.

(b) Flattened output dimension (one scalar per input pixel): 784.

(c) Weights for dense A: 784 * 784 = 614,656.

(d) Weights for shared B: 9.

(e) With biases:

```
A: 614,656 weights + 784 biases = 615,440
B: 9 weights + 1 bias = 10
```

Ratio with biases: 615,440 / 10 = 61,544.

(f) Which assumes locality: B (each output reads only a 3x3 neighbourhood).

(g) Which shares parameters: B (same 9 weights at every position).

(h) Which is naturally translation equivariant (ignoring boundaries): B, the shared stride-1 convolution. A dense map has position-specific weights and no reason to be equivariant.

Question 34 — 64 x 64 image (4096 pixels)

(a) Unrestricted dense linear map weights: 4096 * 4096 = 16,777,216 (about 1.68 x 10^7).

(b) One shared 5 x 5 filter weights: 25.

(c) Ratio: 16,777,216 / 25 = 671,088.64 (about 6.71 x 10^5).

(d) Dense multiply-add contributions: about 4096 * 4096 = 16.8 million.

(e) Convolutional multiply-adds (ignoring boundary): about 4096 * 25 = 102,400.

(f) Why both sparsity and sharing matter:

- Sparsity of connections (locality): each output touches only 25 of 4096 inputs instead of all

4096, so compute drops from O(N^2) to O(N k^2) — 102,400 vs. 16.8 million.

- Parameter sharing: the same 25 weights serve all 4096 outputs, so model size drops from about

17 million to 25, independent of image size. Locality saves compute; sharing saves parameters (and gives equivariance). They are separate.

Question 35 — True or false, with reasons

(a) "Convolution and cross-correlation always produce the same numerical output." FALSE. They coincide only for filters symmetric under reversal, e.g. length-1 or (a, b, a). Otherwise the flip changes the result (see Q2).

(b) "A convolutional filter uses different weights at every image location." FALSE. Weight sharing means the SAME weights are used at every location.

(c) "Increasing image size necessarily increases the number of coefficients in a fixed filter." FALSE. Coefficients depend only on k (and channel counts). Only the number of applications grows.

(d) "A 3x3 convolutional output depends directly only on a local neighbourhood." TRUE. It depends on exactly the 9 pixels of the patch (local connectivity).

(e) "A stride-1 convolutional map is translation equivariant away from boundary effects." TRUE. Q18 proves F(Tx) = T F(x) in the interior.

(f) "Translation equivariance means translating the input leaves the output unchanged." FALSE. That is invariance. Equivariance means the output is translated with the input.

(g) "Max pooling contains trainable weights." FALSE. Max pooling has zero parameters.

(h) "Pooling generally reduces spatial resolution when its stride exceeds one." TRUE. A 2x2 stride-2 pool halves each spatial dimension.

(i) "A convolutional map can use dramatically fewer parameters than an unrestricted dense map." TRUE. Q33: 10 vs. 615,440.

(j) "Weight sharing is closely connected with translation equivariance." TRUE. Sharing the same position-independent filter is exactly what makes a shift of the input equal a shift of the output.

Question 36 — Identify the concept(s)

Core concepts: local connectivity, weight sharing, translation equivariance, pooling.

(a) Same edge detector used top-left and bottom-right: weight sharing; also yields translation equivariance.

(b) An output unit examines only a 3x3 neighbourhood: local connectivity.

(c) A detected feature moves when the object moves: translation equivariance.

(d) A 28x28 feature map reduced to 14x14: pooling (with stride 2).

(e) Far fewer parameters than a dense map: local connectivity + weight sharing (both).

(f) Nearby strong activations summarised by one value: pooling (max pooling).

Question 37 — Pipeline on a 32 x 32 input

Operations: 3x3 convolution, stride 1, padding 1 -> ReLU -> 2x2 max pooling, stride 2. One convolutional output channel.

(a) Shape after convolution:

```
H' = (32 + 2*1 - 3)/1 + 1 = 32,  W' = 32  ->  32 x 32 x 1
```

(Same padding: output equals input spatial size.)

(b) Shape after ReLU: unchanged, 32 x 32 x 1 (pointwise).

(c) Shape after max pooling:

```
(32 - 2)/2 + 1 = 16  ->  16 x 16 x 1
```

(d) Conv trainable parameters: k^2*C_in*C_out + C_out = 9*1*1 + 1 = 10 (9 weights + 1 bias).

(e) ReLU parameters: 0.

(f) Max pooling parameters: 0.

(g) Local weighted feature extraction: convolution.

(h) Pointwise operation: ReLU.

(i) Reduces spatial resolution: max pooling (stride 2).

(j) Can affect exact equivariance through subsampling: max pooling (stride 2). The convolution itself has stride 1 and is equivariant.

Question 38 — Conceptual synthesis: dense vs. convolutional

(a) Prior encoded by local connectivity: relevant image structure is local. An output at a location depends mainly on nearby pixels; distant pixels matter less. Neighbouring pixels are statistically correlated (smoothness/Markov structure of natural images).

(b) Prior encoded by sharing a filter: image statistics are stationary / translation invariant. The same pattern (edge, texture, object part) can occur anywhere, so the same detector should apply everywhere. The detection rule does not depend on absolute position.

(c) How locality reduces computation: each output uses k^2 inputs instead of all N, so cost per output is constant and total cost is O(k^2 N) instead of O(N^2): linear vs. quadratic.

(d) How sharing reduces parameters: one filter (O(k^2) coefficients) is reused at all locations, so parameter count is independent of N (9 or 25 vs. millions).

(e) How sharing leads to equivariance: because the same weights apply at every position, shifting the input is absorbed by shifting the summation index, so F(Tx) = T F(x) (Q18). The response moves with the feature.

(f) What pooling loses: exact spatial position/localisation and fine detail. Many inputs map to one output, so precise coordinates and small distinctions within a window are discarded.

(g) Why that loss is useful: it gives robustness to small shifts, noise and nuisance variations, reduces spatial resolution and computation, and increases the effective receptive field so later layers see larger context. It trades localisation precision for invariance and efficiency.

(h) When a dense map could represent something a shared convolution cannot: when behaviour must be position-dependent (spatially varying) — different processing in different image regions (foveation, fixed spatial attention, position-specific templates, or a learned positional bias per pixel). A single shared filter applies identical weights everywhere, so it cannot encode "do X in the top-left and Y in the bottom-right".

(i) Fundamental reason convolutions suit images: images have two structural symmetries — locality (nearby pixels interact) and stationarity (patterns look the same wherever they occur). A convolution encodes exactly these priors: local receptive fields plus shared weights. This gives large parameter and compute savings, translation equivariance, and good generalisation, which is why convolutional maps are the natural inductive bias for images.

Part 3 — Formula Card, Pitfalls, Self-Test

3.1 Formula card

```
1-D valid output            : n - k + 1
1-D general output          : floor((n + 2p - k)/s) + 1
2-D general output          : floor((H + 2p_h - k_h)/s_h) + 1
                              x floor((W + 2p_w - k_w)/s_w) + 1
Same padding (stride 1)     : p = (k - 1)/2
Conv weights (1 out chan)   : k^2 * C_in
Conv total params           : k^2 * C_in * C_out + C_out
Dense params (d -> m)       : m*d + m
Conv multiplications        : k^2 (H-k+1)(W-k+1)
Dense multiplications (N->N): N^2
Cost ratio conv vs dense    : N / k^2  (dense is N/k^2 times more expensive)
Receptive field             : 1 + L(k - 1); for 3x3: 2L + 1
Pooling output              : floor((n - k)/s) + 1; 2x2 stride 2 halves each side
Max/average pooling params  : 0
Conv vs correlation         : conv = correlation with reversed kernel
```

3.2 Equivariance in one line

```
Equivariance: F(T_d x) = T_d F(x)     output moves with input
Invariance:   F(T_d x) = F(x)         output unchanged

Conv + pointwise nonlinearity  = equivariant.
Strided conv/pooling           = equivariant only to multiples of the stride, interior only.
Pooling                        = local, approximate invariance.
```

3.3 Self-test (answers in brackets)

1. Input length 20, filter 4, stride 2, padding 0 -> output length? (9)
2. 32x32, 3x3 conv, 4 in-channels, 8 out-channels -> params? (9*4*8 + 8 = 296)
3. Same conv at all 1024 positions -> how many distinct weight sets? (1)

4. Two 3x3 stride-1 convs -> RF? (5). Five -> (11).
5. Is a stride-1 conv translation invariant? (No — equivariant)
6. 2x2 stride-2 max pool on 28x28 -> shape? (14x14)
7. Dense 784->784 weights? (614,656). Shared 3x3 weights? (9)
8. Conv vs. xcorr for k = (2, -1, 2)? (equal — palindromic)
9. Zero-sum filter on a constant patch -> response? (0)
10. N = 10^6, k = 3: conv is how many times cheaper than dense? (about 1.11 x 10^5)

All numerical results above were independently verified by direct computation.